Kubron: требования виджетной платформы к веб-версии

Для команды веб-версии · версия 0.1 · 27.10.2026 · на согласование

1. Зачем этот документ

Мы готовим виджетную платформу для экрана Overview: плашки становятся независимыми виджетами со свободной раскладкой, а позже появятся сторонние виджеты из маркетплейса (по аналогии с расширениями VS Code). Полная картина — в документе «Kubron: план разработки виджетной платформы».

Веб-режима это касается в нескольких местах, где решение дешевле принять сейчас, чем переделывать после этапов 2–3. Приоритеты веб-ветки документ не меняет: P0 — небольшие задачи, остальное привязано к уже запланированным этапам.

2. Что такое виджет (минимум контекста)

- Виджет — независимый блок на Overview: манифест (id, размеры, права) и интерфейс.
- **Встроенные** виджеты — доверенные Vue-компоненты в основном окне.
- **Сторонние** — чужой JS. Он выполняется в `<iframe sandbox="allow-scripts">` без `allow-same-origin` (непрозрачный origin) и общается с хостом только через `postMessage`. К `/api/rpc` у него доступа нет: данные и действия выдаёт хост в рамках разрешений, которые виджет заявил, а пользователь подтвердил.

3. Что мы увидели в текущем состоянии

Источники: `docs/handoff.md`, `docs/tickets/stage-1.md`, ADR 0003, 0004, 0007, 0010, 0012 и поиск по ветке `refactor`. Код `internal/server` мы подробно не разбирали, поэтому пункты ниже просим проверить владельцев кода.

1. CSRF-токен доступен JS (ответ Login и `GET /api/session`). Любой скрипт в origin приложения может вызывать RPC с правами пользователя, поэтому сторонний код в этом origin выполнять нельзя.

2. Проверки `Origin`, `Sec-Fetch-Site` (только same-origin/none) и `Content-Type` как раз отсекут запросы из песочницы (у неё `Origin: null`). Это свойство нужно сохранить и закрепить тестами.
3. Заголовка CSP в Go-коде и в `frontend/index.html` мы не нашли.
4. Исходящие запросы сервера по URL пользователя (Helm) не защищены от SSRF — это известный пробел из ваших заметок. Сетевой прокси для виджетов будет опираться на тот же компонент.
5. Ядро пока одно на всех, а адресация событий и владельцы логов/терминалов — блокеры многопользовательского режима (этап 2).

4. Требования

P0 — сделать в ближайшее время (малые задачи); P1 — в рамках этапов 2–3; P2 — по готовности.

R1 (P0). CSP основного приложения

Что. Заголовок `Content-Security-Policy` в `kubron-server` для SPA: сначала `Report-Only`, затем enforcing. Основа: `default-src 'self'`, `object-src 'none'`, `base-uri 'none'`, `frame-ancestors 'none'`, `connect-src 'self'` (с учётом WebSocket), `frame-src` только на путь содержимого виджетов (R3). Для десктопа аналог через AssetServer Wails — наша часть.

Оговорка. Monaco (web workers, inline-стили) и xterm могут потребовать послаблений (`worker-src blob:`, `style-src 'unsafe-inline'`) — нужно проверить.

Приёмка. Режим Report-Only без нарушений на основных экранах (ресурсы, редактор, терминал, Argo, Helm); e2e проверяет наличие заголовка.

R2 (P0). Тесты границы запросов

Что. В контрактные или e2e-тесты добавить отказные случаи для `/api/rpc/*`, `/api/ws`, `/api/terminal/*`, `/api/session`: `Origin: null`, чужой Origin, `Sec-Fetch-Site: cross-site`, нет cookie, нет CSRF. CORS-заголовки на `/api/*` не включать.

Приёмка. Тесты в CI; `GET /api/session` не отдаёт CSRF-токен запросу с `Origin: null`.

R3 (P0 заглушка, P1 полностью). Раздача содержимого сторонних виджетов

Что. Отдельный обработчик `/widgets/{id}/{version}/…` (или отдельный origin — порт или поддомен; выбор за вами с учётом `--trust-proxy` и TLS). Требования к нему:

- не принимает и не выставляет cookie, не даёт доступа к `/api/*`;
- заголовки: `Content-Security-Policy: default-src 'none'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'none'; base-uri 'none'; form-action 'none'`, `X-Content-Type-Options: nosniff`, кэш по версии;
- `Access-Control-Allow-Origin: *` только для статики виджета (модульные скрипты из песочницы грузятся в CORS-режиме, у документа `Origin: null`) — проверить на практике;
- защита от обхода путей; источник файлов — `<data-dir>/widgets/`.

Почему заголовком. Атрибут `csp` у iframe, насколько нам известно, работает только в Chromium, а десктоп на macOS и Linux — WebKit. Надёжно только CSP из ответа сервера.

Приёмка. Тест заголовков на заглушке; e2e с тестовым «враждебным» виджетом: из него недоступны `/api/*`, cookie и `localStorage` приложения.

R4 (P1). Хранилище UI-настроек пользователя

Что. В интерфейсе `Store` (ADR-0007) предусмотреть KV на пару «пользователь + кластер»: ключ до 128 символов, значение JSON до 64 КБ, версия схемы, `updatedAt`. Методы вроде `GetUIPref`, `SetUIPref`, `DeleteUIPref` — с `authz` и классификацией в `internal/server/methods.go`. Пока раскладка живёт в `localStorage`; позже переедет на сервер, чтобы следовать за пользователем между браузерами.

Приёмка. Контрактный тест для файловой и PostgreSQL-реализаций; чужие записи недоступны.

R5 (P1). Серверный реестр установленных виджетов

Что. Запись: id, версия, хэш, издатель, выданные разрешения, enabled, кто и когда установил. Управляет админ (новое право, например `widgets.manage`, — синхронно в `internal/auth/permissions.go` и `types/auth.ts`). Область установки (организация или команда) решить вместе с ADR-0012.

R6 (P1). Безопасный HTTP-клиент для исходящих запросов

Что. Общий пакет для Helm и будущего прокси виджетов: запрет loopback, private, link-local (в том числе `169.254.169.254`) и unique-local IPv6; проверка IP в момент соединения (против DNS-rebinding); проверка на каждом редиректе; только http/https; лимиты размера и времени; опциональный allowlist доменов.

Приёмка. Таблица тестов на обходы.

R7 (P1). Адресация событий по сессии

Уже в плане этапа 2. Виджеты получают живые данные через хост, поэтому события одного пользователя не должны попадать другому. Новых работ нет, фиксируем зависимость.

R8 (P1). Аудит

Поле `via` в записи аудита (например, `widget:acme.certs@1.2.0`), чтобы отличать действия виджета от действий пользователя; события установки, включения и смены разрешений.

R9 (P2). Capabilities и политика сервера

`GetCapabilities` дополнить флагами `widgets: { builtin, thirdParty, marketplace }` (в духе ADR-0010) и флагами сервера `--widgets=off|builtin|installed|marketplace`, `--widgets-allowlist`, `--widgets-dir`. Безопасное значение по умолчанию для веба: сторонние виджеты выключены.

R10 (P2). Без новых «открытых» эндпоинтов

Новые HTTP-эндпоинты не должны быть доступны без cookie и CSRF и не должны включать CORS. Держатель CSRF-токена — только основное окно (хост виджетов).

5. Чего мы не просим

- Менять слой `platform/` и транспорт.
- Ждать нас: P0 не зависят от виджетного кода.
- Реализовывать песочницу и маркетплейс в веб-ветке — это наша работа после этапа 2.

6. Привязка к этапам

Требование	Когда
R1, R2, заглушка R3	сейчас, параллельно этапу 2
R7	этап 2
R4, R5, R8, полный R3	вместе с ADR-0007 и этапом 3
R6	до включения сетевого прокси виджетов
R9, R10	при реализации виджетной платформы

7. Модель угроз (кратко)

Угроза	Мера	Где
Виджет вызывает RPC от имени пользователя	песочница без `allow-same-origin`, проверка Origin, токен только у хоста	R1–R3
Вывод данных наружу	CSP `connect-src 'none'`, сеть только через хост-прокси с allowlist	R3, R6
SSRF через прокси	безопасный HTTP-клиент	R6
Повышение прав	права = пересечение разрешений виджета и прав пользователя; запись — отдельное разрешение (наша часть)	—
Подмена пакета, вредоносная версия	подпись, хэши, список отзыва (наша часть)	—
Действия виджета неотличимы от действий пользователя	поле `via` в аудите	R8

8. Вопросы к команде веба

1. Отдельный origin для содержимого виджетов или путь на том же origin? Мы за отдельный origin, если он не усложняет `--trust-proxy` и TLS.
2. Где в ADR-0007 живёт KV UI-настроек: отдельная сущность или общая таблица preferences?

3. Входит ли `widgets.manage` в базовые роли или остаётся только у администратора?
4. Реально ли уложить R1–R3 до начала этапа 3?